

1. Reactive Programming และ Blocking Programming แตกต่างกันอย่างไร

1.1 Blocking Programming

Blocking Programming คือรูปแบบการทำงานที่ Thread จะต้องหยุดรอผลลัพธ์จากการทำงานหนึ่งก่อนจึงจะสามารถทำงานต่อได้

ตัวอย่างเช่น หากโปรแกรมต้องเรียกข้อมูลจากฐานข้อมูล โปรแกรมจะส่งคำสั่งไปยังฐานข้อมูล แล้ว Thread จะหยุดรอจนกว่าจะได้รับข้อมูลกลับมา

Java

```
Product product = database.findById("1");  
  
System.out.println(product);
```

ในระหว่างที่กำลังค้นหาข้อมูล Thread จะไม่สามารถนำไปทำงานอื่นได้ หากมีผู้ใช้งานจำนวนมาก อาจทำให้เกิดปัญหา Thread ไม่เพียงพอ และระบบตอบสนองช้าลง

1.2 Reactive Programming

Reactive Programming คือแนวทางการเขียนโปรแกรมที่ทำงานกับข้อมูลแบบ Asynchronous และ Data Stream โดยไม่ต้องให้ Thread หยุดรอผลลัพธ์

แทนที่จะคืนค่าข้อมูลโดยตรง ระบบจะคืนค่าเป็น Publisher เช่น `Mono` หรือ `Flux` เพื่อบอกว่า เมื่อข้อมูลพร้อมแล้วจึงค่อยส่งข้อมูลออกมา

Java

```
Mono<Product> product = repository.findById("1");
```

ในตัวอย่างนี้ `Mono<Product>` ยังไม่ได้หมายความว่าได้ข้อมูลสินค้าแล้ว แต่เป็นตัวแทนของกระบวนการที่จะส่งข้อมูลสินค้าออกมาในอนาคต

1.3 ตารางเปรียบเทียบ

หัวข้อ	Blocking Programming	Reactive Programming
การทำงานของ Thread	หยุดรอผลลัพธ์	ไม่หยุดรอ
การจัดการข้อมูล	คืนค่าข้อมูลโดยตรง	คืนค่าเป็น Mono หรือ Flux
การใช้ทรัพยากร	ใช้ Thread จำนวนมาก	ใช้ Thread ได้อย่างมีประสิทธิภาพ
ความสามารถรองรับผู้ใช้	รองรับผู้ใช้จำนวนมากได้น้อยกว่า	รองรับผู้ใช้จำนวนมากได้ดีกว่า
ตัวอย่าง	JDBC, JPA แบบ Blocking	Spring WebFlux, WebClient
การเรียกใช้	ได้ข้อมูลทันที	ได้ข้อมูลเมื่อมีการ Subscribe
คำสั่งที่ควรหลีกเลี่ยง	-	ห้ามใช้ .block()

1.4 การนำมาใช้ในโปรเจกต์

โปรเจกต์นี้ใช้ Spring WebFlux ซึ่งเป็น Framework สำหรับการพัฒนา Web Application แบบ Reactive โดย Controller จะคืนค่าเป็น **Mono<Product>**, **Flux<Product>** หรือ **Mono<Void>** แทนการคืนค่า **Product** โดยตรง

ตัวอย่าง Controller แบบ Reactive:

```
Java
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}
```

ในโค้ดนี้ Controller ไม่ได้ใช้ **.block()** เพื่อรอข้อมูล แต่คืน **Flux<Product>** ให้ Spring WebFlux จัดการการส่งข้อมูลให้ผู้ใช้เอง

ดังนั้น Thread จึงไม่ต้องหยุดรอผลลัพธ์ และสามารถนำไปจัดการ Request อื่นได้

2. Mono และ Flux ใช้กรณีไหน และทำไม

ใน Project Reactor ซึ่งเป็นส่วนสำคัญของ Spring WebFlux มี Reactive Type ที่ใช้บ่อย 2 ชนิด คือ **Mono** และ **Flux**

2.1 Mono

Mono<T> ใช้แทนข้อมูลจำนวน 0 หรือ 1 ค่า

None

Mono<T> = 0 หรือ 1 ค่า

ตัวอย่างการใช้ **Mono**:

Java

```
Mono<Product> product = repository.findById("1");
```

เหตุผลที่ใช้ **Mono<Product>** เพราะการค้นหาสินค้าด้วย ID จะมีผลลัพธ์ได้เพียงอย่างเดียว คือ

- พบสินค้า 1 รายการ
- ไม่พบสินค้า หรือมีผลลัพธ์เป็น 0 รายการ

ตัวอย่าง Method ที่ใช้ **Mono** ในโปรเจกต์:

Java

```
public Mono<Product> getById(String id) {  
    return repository.findById(id)  
        .switchIfEmpty(  
            Mono.error(  
                new RuntimeException(  
                    "Product not found: " + id  
                )  
            )  
        )  
};
```

```
}
```

นอกจากนี้ยังใช้ **Mono** ในกรณีต่อไปนี้:

- สร้างสินค้า 1 รายการ
- ค้นหาสินค้าตาม ID
- คำนวณราคาหลังส่วนลด
- ลบสินค้าโดยใช้ **Mono<Void>**

2.2 Flux

Flux<T> ใช้แทนข้อมูลจำนวน 0 ถึงหลายค่า

None

Flux<T> = 0 ถึงหลายค่า

ตัวอย่างการใช้ **Flux**:

Java

```
Flux<Product> products = repository.findAll();
```

เหตุผลที่ใช้ **Flux<Product>** เพราะการค้นหาสินค้าทั้งหมดอาจพบสินค้าได้หลายรายการ

ตัวอย่าง Method ที่ใช้ **Flux**:

Java

```
public Flux<Product> getAll() {  
    return repository.findAll();  
}
```

นอกจากนี้ยังใช้ **Flux** ในกรณีต่อไปนี้:

- ดึงสินค้าทั้งหมด
- ค้นหาสินค้าตาม Category
- รับข้อมูลที่มีหลายรายการ
- อ่านข้อมูลแบบ Stream

2.3 Mono

`Mono<Void>` ใช้เมื่อมีการทำงานบางอย่างเสร็จสิ้น แต่ไม่ต้องการส่งข้อมูลกลับไป

ตัวอย่างเช่น การลบสินค้า:

```
Java
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}
```

2.4 ตารางสรุป Mono และ Flux

Reactive Type	จำนวนข้อมูล	ตัวอย่างการใช้งาน
<code>Mono<Product></code>	0 หรือ 1	ค้นหาสินค้าตาม ID
<code>Flux<Product></code>	0 ถึงหลายรายการ	ค้นหาสินค้าทั้งหมด
<code>Mono<Double></code>	0 หรือ 1	ราคาหลังส่วนลด
<code>Mono<Void></code>	ไม่มีข้อมูลตอบกลับ	ลบสินค้า

2.5 เหตุผลที่ต้องเลือกใช้ให้ถูกต้อง

การเลือกใช้ `Mono` และ `Flux` ให้ตรงกับจำนวนข้อมูลทำให้โค้ดมีความชัดเจน และทำให้ Spring WebFlux สามารถจัดการ Stream ของข้อมูลได้อย่างถูกต้อง

ตัวอย่าง:

Java

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}
```

Method นี้ใช้ `Mono<Product>` เพราะ URL นี้ค้นหาสินค้าเพียง 1 รายการ

Java

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}
```

Method นี้ใช้ `Flux<Product>` เพราะ URL นี้สามารถค้นหาสินค้าหลายรายการ

3. WebClient และการอธิบาย Method Chain

`WebClient` คือเครื่องมือของ Spring WebFlux ที่ใช้เรียก REST API ผ่าน HTTP แบบ Reactive และ Non-blocking

ตัวอย่างการสร้าง WebClient:

Java

```
private final WebClient client =
    WebClient.create("http://localhost:8080");
```

คำสั่งนี้สร้าง WebClient ที่มี Base URL เป็น

None

`http://localhost:8080`

ดังนั้นหากเรียก URI เป็น

None

`/products/1`

URL เต็มจะเป็น

None

`http://localhost:8080/products/1`

3.1 `.get()`

ใช้กำหนด HTTP Method เป็น GET

Java

`client.get()`

เหมาะสำหรับการดึงข้อมูล เช่น

None

`GET /products`

`GET /products/1`

3.2 `.uri()`

ใช้กำหนด URL หรือ Path ที่ต้องการเรียก

Java

`.uri("/products/{id}", id)`

ถ้า `id` มีค่าเป็น `"1"` URL จะกลายเป็น

None

/products/1

ตัวอย่าง:

Java

```
client.get()  
    .uri("/products/{id}", id)
```

3.3 .retrieve()

ใช้เริ่มรับข้อมูลจาก HTTP Response

Java

```
.retrieve()
```

หลังจากเรียก `.retrieve()` แล้ว จึงสามารถกำหนดได้ว่าจะให้แปลง Response เป็น `Mono` หรือ `Flux`

3.4 .bodyToMono()

ใช้แปลง Response Body เป็น `Mono<T>`

เหมาะกับการรับข้อมูล 0 หรือ 1 รายการ

Java

```
.bodyToMono(Product.class)
```

ตัวอย่างเต็ม:

Java

```
public Mono<Product> getProductById(String id) {  
    return client.get()
```

```
.uri("/products/{id}", id)
.retrieve()
.bodyToMono(Product.class);
}
```

ลำดับการทำงานคือ

```
None
เรียก GET
↓
กำหนด URI
↓
รับ Response
↓
แปลง JSON เป็น Mono<Product>
```

3.5 .bodyToFlux()

ใช้แปลง Response Body เป็น `Flux<T>`

เหมาะกับการรับข้อมูลหลายรายการ

```
Java
.bodyToFlux(Product.class)
```

ตัวอย่างเต็ม:

```
Java
public Flux<Product> getAllProducts() {
    return client.get()
```

```
.uri("/products")  
.retrieve()  
.bodyToFlux(Product.class);  
}
```

3.6 การเรียก POST ด้วย WebClient

การส่งข้อมูลด้วย POST ใช้ `.post()` และ `.bodyValue()`

```
Java  
  
public Mono<Product> createProduct(Product product) {  
    return client.post()  
        .uri("/products")  
        .bodyValue(product)  
        .retrieve()  
        .bodyToMono(Product.class);  
}
```

ลำดับการทำงานคือ

```
None  
เรียก POST  
↓  
กำหนด URI  
↓  
ส่ง Product ใน Request Body  
↓  
รับผลลัพธ์ Product ที่ถูกสร้าง
```

3.7 การเรียก DELETE ด้วย WebClient

Java

```
public Mono<Void> deleteProduct(String id) {  
    return client.delete()  
        .uri("/products/{id}", id)  
        .retrieve()  
        .bodyToMono(Void.class);  
}
```

เนื่องจากการลบไม่จำเป็นต้องส่งข้อมูลกลับ จึงใช้ `Mono<Void>`

3.8 การ Chain Operators หลังจาก WebClient

ตัวอย่างการใช้ `map()`, `defaultIfEmpty()` และ `subscribe()`:

Java

```
getProductById("1")  
    .map(Product::getName)  
    .defaultIfEmpty("ไม่พบสินค้า")  
    .subscribe(  
        name -> System.out.println(  
            "Product name: " + name  
        ),  
        error -> System.err.println(  
            "Error: " + error.getMessage()  
        )  
    );
```

คำอธิบาย:

Java

```
.map(Product::getName)
```

แปลงข้อมูลจาก **Product** ให้เหลือเฉพาะชื่อสินค้า

```
Java  
  
.defaultIfEmpty("ไม่พบสินค้า")
```

กำหนดข้อความสำรองกรณีไม่มีข้อมูล

```
Java  
  
.subscribe(...)
```

เริ่มต้นการทำงานของ Reactive Stream และรับผลลัพธ์ที่ส่งกลับมา

อย่างไรก็ตาม ใน Controller ไม่ควรเรียก **subscribe()** เอง เพราะ Spring WebFlux จะจัดการ Subscribe ให้โดยอัตโนมัติ

4. อธิบาย Code ของ Repository, Service และ Controller

โปรเจกต์นี้แบ่งโค้ดออกเป็นหลาย Layer เพื่อให้แต่ละส่วนมีหน้าที่ชัดเจน ได้แก่ Model, Repository, Service, Controller และ Client

```
None  
Client  
↓  
Controller  
↓  
Service  
↓  
Repository  
↓  
In-memory Storage
```

4.1 Repository Layer

Repository มีหน้าที่จัดการข้อมูลสินค้าโดยตรง ในโปรเจกต์นี้ไม่ได้ใช้ฐานข้อมูลจริง แต่ใช้ `ConcurrentHashMap` เป็นที่เก็บข้อมูลชั่วคราว

โค้ด ProductRepository

```
Java

package com.example.lab10.repository;

import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ProductRepository {

    /**
     * ใช้ ConcurrentHashMap เป็นพื้นที่เก็บข้อมูลสินค้า
     *
     * * Key คือ id ของสินค้า
     * * Value คือ Product
     */

    private final Map<String, Product> store =
        new ConcurrentHashMap<>();

    /**
     * สร้างข้อมูลตัวอย่างเริ่มต้น
     */

    public ProductRepository() {
```

```
store.put(  
  "1",  
  new Product(  
    "1",  
    "iPhone 15 Pro",  
    "Electronics",  
    "Apple",  
    50,  
    39900.0,  
    "MEMBER"  
  )  
);
```

```
store.put(  
  "2",  
  new Product(  
    "2",  
    "MacBook Air M3",  
    "Electronics",  
    "Apple",  
    20,  
    49900.0,  
    "NONE"  
  )  
);
```

```
store.put(  
  "3",  
  new Product(  

```

```

        "3",
        "Samsung Galaxy S24",
        "Electronics",
        "Samsung",
        30,
        29900.0,
        "SEASONAL "
    )
);
}

/*
 * ค้นหาสินค้าตาม id
 *
 * ถ้าพบสินค้า จะคืน Mono.just(product)
 * ถ้าไม่พบสินค้า จะคืน Mono.empty()
 */
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    if (product == null) {
        return Mono.empty();
    }

    return Mono.just(product);
}

/*
 * ค้นหาสินค้าทั้งหมด

```

```

*
* store.values() เป็น Collection<Product>
* จึงใช้ Flux.fromIterable() แปลงเป็น Flux
*/
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

/*
* บันทึกสินค้าใหม่หรือแก้ไขสินค้าที่มี id เดิม
*/
public Mono<Product> save(Product product) {
    return Mono.fromSupplier(() -> {
        store.put(product.getId(), product);
        return product;
    });
}

/*
* ลบสินค้าตาม id
*
* ใช้ Mono<Void> เพราะไม่ต้องการส่งข้อมูลสินค้ากลับ
*/
public Mono<Void> deleteById(String id) {
    return Mono.fromRunnable(() -> store.remove(id));
}

/*
* ค้นหาสินค้าตาม category

```

```

*
* ใช้ filter เพื่อเลือกเฉพาะสินค้าที่ category ตรงกัน
*/
public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(product ->
            product.getCategory() != null
                && product.getCategory()
                    .equalsIgnoreCase(category)
        );
}
}

```

หน้าที่ของแต่ละ Method

Method	หน้าที่	Return Type
findById()	ค้นหาสินค้าตาม ID	Mono<Product>
findAll()	ค้นหาสินค้าทั้งหมด	Flux<Product>
save()	บันทึกสินค้า	Mono<Product>
deleteById()	ลบสินค้า	Mono<Void>
findByCategory()	ค้นหาตาม Category	Flux<Product>

4.2 Service Layer

Service ทำหน้าที่เป็นส่วนกลางระหว่าง Controller และ Repository โดยเก็บ Business Logic ของระบบ เช่น

- ตรวจสอบว่าพบสินค้าหรือไม่

- สร้าง ID ใหม่
- ตรวจสอบชื่อสินค้า
- คำนวณราคาหลังส่วนลด

โค้ด ProductService

Java

```
package com.example.lab10.service;

import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.UUID;

@Service
public class ProductService {

    private final ProductRepository repository;

    /**
     * Constructor Injection
     *
     * รับ ProductRepository จาก Spring
     */
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }
}
```

```

/*
 * ค้นหาสินค้าตาม id
 *
 * ถ้าไม่พบสินค้า จะสร้าง Error ด้วย switchIfEmpty()
 */
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException(
                    "Product not found: " + id
                )
            )
        );
}

```

```

/*
 * ดึงสินค้าทั้งหมด
 */
public Flux<Product> getAll() {
    return repository.findAll();
}

```

```

/*
 * บันทึกสินค้า
 */
public Mono<Product> save(Product product) {

    /*

```

** Mono.defer ทำให้คำสั่งด้านในทำงานเมื่อมีการ Subscribe*

**/*

return Mono.defer(() -> {

*/**

** ถ้าไม่มี id ให้สร้าง id ใหม่ด้วย UUID*

**/*

if (product.getId() == null
|| product.getId().isBlank()) {

product.setId(
UUID.randomUUID().toString()

);

}

*/**

** filter ใช้ตรวจสอบว่าชื่อสินค้าไม่เป็นค่าว่าง*

**/*

return Mono.just(product)
.filter(p ->
p.getName() != null
&& !p.getName().isBlank()
)

*/**

** ถ้าชื่อสินค้าว่าง ให้ส่ง Error*

**/*

.switchIfEmpty(
Mono.error(

```

        new IllegalArgumentException(
            "Product name is required"
        )
    )
)

/*
 * flatMap ใช้เรียก repository.save()
 * เนื่องจาก save() คืนค่าเป็น Mono<Product>
 */
.flatMap(repository::save);
});
}

/*
 * ลบสินค้า
 */
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

/*
 * ค้นหาสินค้าตาม category
 */
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

/*

```

```

* คำนวณราคาหลังส่วนลด
*
* map ใช้แปลง Product เป็น Double
*/
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(Product::getDiscountedPrice);
}
}

```

Operators ที่ใช้ใน Service

```

Java
.switchIfEmpty(...)

```

ใช้จัดการกรณีที่ไม่มีข้อมูล เช่น ไม่พบสินค้า

```

Java
.filter(...)

```

ใช้ตรวจสอบหรือกรองข้อมูล

```

Java
.flatMap(...)

```

ใช้เชื่อมต่อกับ Method ที่คืนค่าเป็น **Mono** หรือ **Flux**

```

Java
.map(...)

```

ใช้แปลงข้อมูลจากชนิดหนึ่งเป็นอีกชนิดหนึ่ง

ตัวอย่าง:

```
Java  
Mono<Product>
```

ถูกแปลงเป็น

```
Java  
Mono<Double>
```

ด้วยคำสั่ง

```
Java  
.map(Product::getDiscountedPrice)
```

4.3 Controller Layer

Controller ทำหน้าที่รับ HTTP Request จากผู้ใช้ และส่งต่อไปยัง Service

Controller ไม่ควรเขียน Business Logic มากเกินไป แต่ควรทำหน้าที่รับ Request และคืนค่า Reactive Type

โค้ด ProductController

```
Java  
  
package com.example.lab10.controller;  
  
import com.example.lab10.model.Product;  
import com.example.lab10.service.ProductService;  
import org.springframework.web.bind.annotation.*;  
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    /*
     * Constructor Injection
     */
    public ProductController(ProductService service) {
        this.service = service;
    }

    /*
     * GET /products/{id}
     *
     * ค้นหาสินค้า 1 รายการจาก id
     */
    @GetMapping("/{id}")
    public Mono<Product> getById(
        @PathVariable String id
    ){
        return service.getById(id);
    }

    /*
     * GET /products
```

```

*
* ค้นหาสินค้าทั้งหมด
*/

@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

/*
* POST /products
*
* รับ Product จาก Request Body
* แล้วส่งต่อให้ Service บันทึก
*/

@PostMapping
public Mono<Product> save(
    @RequestBody Product product
){
    return service.save(product);
}

/*
* DELETE /products/{id}
*
* ลบสินค้าตาม id
*/

@DeleteMapping("/{id}")
public Mono<Void> delete(
    @PathVariable String id

```

```

    ){
        return service.delete(id);
    }

    /**
     * GET /products/category/{category}
     *
     * ค้นหาสินค้าตาม category
     */
    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(
        @PathVariable String category
    ){
        return service.getByCategory(category);
    }

    /**
     * GET /products/{id}/price
     *
     * แสดงราคาสินค้าหลังหักส่วนลด
     */
    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(
        @PathVariable String id
    ){
        return service.getDiscountedPrice(id);
    }
}

```

คำอธิบาย Annotation

Java

@RestController

ระบุว่า Class นี้ทำหน้าที่เป็น REST Controller

Java

@RequestMapping("/products")

กำหนด Base URL ของทุก Method ใน Class นี้

Java

@GetMapping

ใช้รับ HTTP GET Request

Java

@PostMapping

ใช้รับ HTTP POST Request

Java

@DeleteMapping

ใช้รับ HTTP DELETE Request

Java

@PathVariable

ใช้รับค่าที่อยู่ใน URL เช่น

None

/products/1

ค่า 1 จะถูกส่งเข้าไปในตัวแปร id

Java

@RequestBody

ใช้รับข้อมูล JSON จาก Body ของ Request แล้วแปลงเป็น Object **Product**

4.4 เหตุผลที่ Controller ไม่ใช่ **.block()**

โค้ดที่ไม่ควรใช้:

Java

@GetMapping

```
public List<Product> getAll() {  
    return service.getAll().collectList().block();  
}
```

โค้ดนี้เป็น Blocking เพราะใช้ **.block()** เพื่อรอผลลัพธ์

โค้ดที่ถูกต้อง:

Java

@GetMapping

```
public Flux<Product> getAll() {  
    return service.getAll();  
}
```

การคืนค่า **Flux<Product>** โดยตรงทำให้ Spring WebFlux สามารถจัดการการ Subscribe และส่งข้อมูลแบบ Non-blocking ได้เอง

สรุป

ระบบนี้ใช้แนวคิด Reactive Programming โดยแบ่งหน้าที่ของโปรแกรมออกเป็น Layer อย่างชัดเจน Repository รับผิดชอบการจัดเก็บข้อมูล Service รับผิดชอบ Business Logic และ Controller รับผิดชอบการติดต่อกับผู้ใช้ผ่าน REST API

การใช้ Mono, Flux, map, filter, flatMap และ WebClient ทำให้ระบบสามารถจัดการข้อมูลแบบ Asynchronous และ Non-blocking ได้ โดยไม่ต้องใช้ .block() หรือทำให้ Thread หยุดรอผลลัพธ์

